

Alunos: Bernardo Ghinato Goelzer, Igor Grams Broering & Leonardo Felipe Lemos.

1. Requisitos Funcionais

Os requisitos funcionais descrevem todas as funcionalidades que o sistema deve executar. Cada requisito possui um identificador único (RF-XX) e um nível de prioridade: Alta, Média ou Baixa.

1.1 Cadastro e Autenticação

ID	Requisito	Descrição	Prioridade
RF-01	Cadastro de usuário	O sistema deve permitir o registro de novos usuários com nome completo, e-mail e senha (mínimo 8 caracteres com letras e números). O back-end Python deve validar os dados via Pydantic e enviar e-mail de confirmação antes de ativar a conta.	Alta

ID	Requisito	Descrição	Prioridade
RF-02	Login com e-mail e senha	O sistema deve autenticar usuários via e-mail e senha. O back-end deve gerar um token JWT com expiração configurável usando a biblioteca python-jose. Após 5 tentativas falhas, a conta deve ser bloqueada por 15 minutos.	Alta

ID	Requisito	Descrição	Prioridade
RF-03	Recuperação de senha	O sistema deve oferecer recuperação de senha por e-mail. O back-end Python deve gerar um token único de redefinição com validade de 30 minutos, enviado via biblioteca smtplib ou serviço de e-mail transacional.	Alta

ID	Requisito	Descrição	Prioridade
RF-04	Edição de perfil	O usuário deve poder editar nome, foto, e-mail, senha e preferências de acessibilidade. A rota Python correspondente deve exigir autenticação e validar os novos dados via Pydantic antes de persistir.	Média

1.2 Gerenciamento de Tarefas

ID	Requisito	Descrição	Prioridade
RF-05	Criação de tarefa	O sistema deve permitir criar tarefas com: título, descrição, categoria, prioridade (baixa/média/alta/urgente), data de vencimento e tempo estimado. A rota POST /tasks do back-end Python deve salvar no PostgreSQL via SQLAlchemy.	Alta

ID	Requisito	Descrição	Prioridade
RF-06	Edição de tarefa	O usuário deve poder editar todos os campos de uma tarefa existente. A rota PATCH /tasks/{id} deve verificar se o recurso pertence ao usuário autenticado antes de aplicar as alterações.	Alta

ID	Requisito	Descrição	Prioridade
RF-07	Exclusão de tarefa	O usuário deve poder excluir tarefas. A exclusão lógica (soft delete) deve ser aplicada, movendo o item para lixeira. A rota DELETE /tasks/{id} deve impedir exclusão de tarefas de outros usuários.	Média

ID	Requisito	Descrição	Prioridade
RF-08	Conclusão de tarefa	O usuário deve marcar tarefas como concluídas com um único clique. O back-end deve registrar o timestamp de conclusão e atualizar as métricas do usuário no banco.	Alta

ID	Requisito	Descrição	Prioridade
RF-09	Subtarefas	O sistema deve suportar subtarefas vinculadas a uma tarefa pai. O progresso da tarefa pai é calculado automaticamente pelo back-end como (subtarefas concluídas / total) × 100.	Alta

ID	Requisito	Descrição	Prioridade
RF-10	Filtragem e ordenação de tarefas	A rota GET /tasks deve aceitar parâmetros de query para filtrar por status, prioridade, categoria e data, e ordenar por qualquer campo. A lógica de filtragem deve ser implementada com SQLAlchemy queries dinâmicas.	Média

ID	Requisito	Descrição	Prioridade
RF-11	Categorização de tarefas	O sistema deve oferecer categorias padrão (Estudos, Trabalho, Saúde, Pessoal) e permitir categorias personalizadas. Cada categoria possui nome e cor, armazenados na tabela categories do banco.	Média

1.3 Agenda e Rotina

ID	Requisito	Descrição	Prioridade
RF-12	Agenda diária	O sistema deve exibir todos os eventos e tarefas do dia em formato de linha do tempo. A rota GET /agenda/daily?date=YYYY-MM-DD deve retornar os itens ordenados por horário.	Alta

ID	Requisito	Descrição	Prioridade
RF-13	Visualização semanal	O sistema deve oferecer visão semanal da agenda. A rota GET /agenda/weekly?start=YYYY-MM-DD deve retornar os itens dos 7 dias seguintes agrupados por data.	Média

ID	Requisito	Descrição	Prioridade
RF-14	Rotina recorrente	O usuário deve poder criar blocos de rotina com recorrência (diária, semanal, dias úteis). O back-end deve usar Celery para recriar automaticamente as tarefas recorrentes nos horários configurados.	Alta

ID	Requisito	Descrição	Prioridade
RF-15	Organização automática do dia	O sistema deve oferecer, mediante solicitação, uma sugestão automática de organização do dia. O back-end Python deve consultar as tarefas pendentes e enviar contexto à API de IA para gerar o plano otimizado.	Alta

1.4 Lembretes e Notificações

ID	Requisito	Descrição	Prioridade
RF-16	Notificações push	O sistema deve enviar notificações push via Firebase Cloud Messaging (FCM). O back-end Python deve usar o SDK firebase-admin para disparar as notificações nos horários calculados.	Alta

ID	Requisito	Descrição	Prioridade
RF-17	Lembretes por prioridade	O sistema deve calcular a antecedência do lembrete conforme a prioridade da tarefa: Urgente = 1h e 15min; Alta = 2h; Média = 1 dia; Baixa = no dia. A lógica deve ser implementada no serviço de notificações Python.	Alta

ID	Requisito	Descrição	Prioridade
RF-18	Filas assíncronas com Celery	Todos os lembretes devem ser enfileirados via Celery com broker Redis. Isso garante que o envio de notificações não bloqueie as requisições da API principal.	Alta

ID	Requisito	Descrição	Prioridade
RF-19	Adiar lembrete	O usuário deve poder adiar um lembrete em 15 minutos diretamente pela notificação. O back-end deve reagendar a tarefa Celery correspondente ao receber a requisição de adiamento.	Média

ID	Requisito	Descrição	Prioridade
RF-20	Lembretes motivacionais	O sistema deve enviar mensagens motivacionais em horários configuráveis pelo usuário. O conteúdo deve ser gerado pela API de IA com base no progresso recente do usuário.	Média

1.5 Chatbot com Inteligência Artificial

ID	Requisito	Descrição	Prioridade
RF-21	Interface de conversação	O sistema deve oferecer um chat com o assistente de IA. O back-end Python deve receber a mensagem do usuário, montar o prompt com contexto e encaminhar à OpenAI API usando a biblioteca openai.	Alta

ID	Requisito	Descrição	Prioridade
RF-22	Contexto personalizado por perfil	O back-end deve injetar automaticamente no system prompt da OpenAI: o perfil do usuário, as tarefas pendentes do dia e as últimas 10 mensagens da conversa, garantindo respostas contextualizadas.	Alta

ID	Requisito	Descrição	Prioridade
RF-23	Decomposição de projetos em tarefas	O chatbot deve decompor projetos descritos em linguagem natural em listas de tarefas e subtarefas. O back-end deve usar function calling da OpenAI para estruturar a resposta e criar as tarefas no banco mediante confirmação do usuário.	Alta

ID	Requisito	Descrição	Prioridade
RF-24	Sugestão de rotina	O chatbot deve gerar rotinas diárias ou semanais personalizadas. O back-end Python deve formatar a resposta da IA e disponibilizá-la para importação direta na agenda.	Alta

ID	Requisito	Descrição	Prioridade
RF-25	Apoio motivacional	O chatbot deve identificar expressões de frustração ou sobrecarga e responder com empatia e estratégias práticas. O back-end não deve armazenar dados clínicos nem simular atendimento psicológico.	Alta

ID	Requisito	Descrição	Prioridade
RF-26	Histórico de conversas	O back-end deve persistir todas as mensagens do chat na tabela chat_messages do PostgreSQL. A rota GET /chat/history deve retornar as conversas paginadas, ordenadas por data.	Média

1.6 Dashboard Analítico

ID	Requisito	Descrição	Prioridade
RF-27	Progresso semanal	A rota GET /dashboard/summary deve retornar: total de tarefas concluídas, em andamento e atrasadas da semana atual, com comparativo em relação à semana anterior. O back-end deve calcular esses valores com queries SQLAlchemy.	Alta

ID	Requisito	Descrição	Prioridade
RF-28	Gráficos de produtividade	O back-end deve disponibilizar endpoints que retornem dados formatados para os gráficos do front-end: distribuição diária, distribuição por categoria e evolução mensal.	Alta

ID	Requisito	Descrição	Prioridade
RF-29	Streak de atividade	O back-end deve calcular e atualizar diariamente a sequência de dias consecutivos com ao menos uma tarefa concluída (streak). O cálculo deve ser executado por um job Celery ao final de cada dia.	Média

ID	Requisito	Descrição	Prioridade
RF-30	Taxa de conclusão e pontualidade	O back-end deve calcular: taxa de conclusão (concluídas / criadas) e taxa de pontualidade (concluídas antes do prazo / total concluídas). Os valores devem ser expostos na rota GET /dashboard/kpis.	Média

1.7 Acessibilidade e Interface

ID	Requisito	Descrição	Prioridade
RF-31	Modo escuro	O sistema deve oferecer alternância entre modo claro e escuro, com a preferência salva no perfil do usuário no banco de dados.	Alta

ID	Requisito	Descrição	Prioridade
RF-32	Ajuste de fonte	O sistema deve permitir ao usuário selecionar entre três tamanhos de fonte (padrão, médio, grande), aplicados globalmente na interface.	Alta

ID	Requisito	Descrição	Prioridade
RF-33	Leitura por voz (TTS)	O sistema deve oferecer leitura em voz alta de tarefas e mensagens do chatbot usando a Web Speech API (front-end), sem custo adicional de back-end.	Média

ID	Requisito	Descrição	Prioridade
RF-34	Interface de baixa sobrecarga cognitiva	A interface deve priorizar hierarquia visual clara, ícones com rótulos, no máximo 3 ações principais por tela e ausência de pop-ups invasivos, reduzindo a carga cognitiva do usuário.	Alta

2. Requisitos Não Funcionais

Os requisitos não funcionais definem os critérios de qualidade que o sistema deve atender, independentemente das funcionalidades específicas.

ID	Categoria	Descrição
RNF-01	Desempenho	O back-end Python deve responder às requisições da API em até 2 segundos para 95% dos casos. Chamadas à OpenAI devem ser assíncronas (async/await com httpx) para não bloquear o servidor FastAPI.
RNF-02	Desempenho	O sistema deve suportar ao menos 1.000 usuários simultâneos. O servidor FastAPI deve ser executado com Uvicorn em modo assíncrono, e o banco de dados deve usar pool de conexões via SQLAlchemy AsyncEngine.
RNF-03	Segurança	Todas as comunicações devem usar HTTPS/TLS. Senhas devem ser armazenadas com hash usando a biblioteca passlib (bcrypt). Nenhuma credencial deve ser exposta em logs ou respostas de API.
RNF-04	Segurança	O back-end Python deve implementar proteções contra as principais vulnerabilidades OWASP: validação de entrada com Pydantic, prevenção de SQL Injection via ORM, rate limiting com slowapi e proteção CORS configurada corretamente.
RNF-05	Privacidade (LGPD)	O sistema deve estar em conformidade com a LGPD. O usuário deve poder solicitar exportação e exclusão de seus dados. O consentimento deve ser registrado no banco no momento do cadastro.
RNF-06	Usabilidade	A criação de uma tarefa deve ser concluída em no máximo 4 interações. A interface deve ser intuitiva para usuários com pouca experiência tecnológica.
RNF-07	Responsividade	A interface deve funcionar corretamente em telas entre 320px e 2560px de largura. O layout deve seguir abordagem mobile-first.
RNF-08	Acessibilidade	O sistema deve atender ao nível AA das diretrizes WCAG 2.1: contraste mínimo de 4,5:1, todos os elementos acessíveis por teclado e compatibilidade com leitores de tela.
RNF-09	Disponibilidade	O sistema deve ter disponibilidade mínima de 99% mensalmente. Erros não tratados devem ser capturados por middleware global do FastAPI e registrados em sistema de logs.
RNF-10	Manutenibilidade	O código Python deve seguir PEP 8, ter cobertura de testes automatizados (pytest) de ao menos 70% e estar documentado com docstrings. A API deve ser documentada automaticamente via Swagger UI (embutido no FastAPI).
RNF-11	Portabilidade	O back-end Python deve ser executado dentro de contêiner Docker. O arquivo requirements.txt deve listar todas as dependências com versões fixadas para garantir reprodutibilidade do ambiente.
RNF-12	Compatibilidade	A aplicação web deve funcionar nas duas versões mais recentes do Chrome, Firefox, Safari e Edge. O app mobile deve suportar Android 10+ e iOS 14+.

3. Regras de Negócio

As regras de negócio definem as políticas que governam o comportamento do sistema e devem ser implementadas na camada de serviços do back-end Python.

ID	Regra	Descrição
RN-01	Métricas apenas de tarefas concluídas	Somente tarefas com status 'concluída' são computadas nas métricas do dashboard. Tarefas excluídas (soft delete) não entram em nenhum cálculo.
RN-02	Prioridade define antecedência do lembrete	Urgente: lembretes com 1h e 15min de antecedência + alerta no vencimento. Alta: 2h. Média: 1 dia. Baixa: no próprio dia. Essa lógica deve estar centralizada em um módulo Python reutilizável.
RN-03	IA adapta resposta ao perfil	O system prompt enviado à OpenAI deve variar conforme o perfil do usuário: tom mais direto e listas curtas para TDAH; linguagem acolhedora para ansiedade. O perfil é injetado automaticamente pelo back-end.
RN-04	Lembrete inteligente requer histórico	O modo adaptativo de lembretes só é ativado após o usuário ter registrado ao menos 5 dias de atividade. Antes disso, o sistema usa os horários padrão definidos pela prioridade (RN-02).
RN-05	Progresso calculado pelas subtarefas	O percentual de progresso de uma tarefa pai é: $(\text{subtarefas concluídas} \div \text{total de subtarefas}) \times 100$, arredondado para inteiro. Tarefas sem subtarefas ficam em 0% até serem concluídas diretamente.
RN-06	Limite diário de tarefas agendadas	A organização automática do dia não deve agendar mais de 8 horas de atividades. O algoritmo Python deve respeitar os horários de descanso definidos nas preferências do usuário ao distribuir as tarefas.
RN-07	Contexto da IA limitado a 10 mensagens	O back-end deve incluir no máximo as últimas 10 mensagens do histórico ao montar o contexto para a OpenAI, controlando o tamanho do prompt e o custo da API.
RN-08	Exclusão de dados conforme LGPD	Ao solicitar exclusão da conta, todos os dados pessoais devem ser anonimizados ou removidos em até 30 dias. O back-end Python deve executar essa rotina via job Celery agendado.
RN-09	Streak reiniciado sem conclusão no dia	A sequência de dias (streak) é incrementada quando ao menos uma tarefa é concluída no dia. Se nenhuma tarefa for concluída, a sequência é zerada. O cálculo é executado pelo job Celery de encerramento do dia.
RN-10	Modo foco filtra notificações	Com o modo foco ativo, apenas notificações de tarefas com prioridade 'Urgente' são enviadas. As demais são enfileiradas e entregues em lote ao término do modo foco.